

Namespace Collision Probability, Worked Out by Hand

Why a small memory keyspace leaks tenant data far sooner than you would guess — the birthday problem, applied to the `(tenant_id, user_id)` namespace.

What this document does. It takes the one safety rule from Module 2.11 — “*always namespace long-term memory by (tenant, user), never global*” — and asks the quantitative question hiding underneath it: if two tenants land in the *same* namespace bucket, tenant A’s memories can surface in tenant B’s session (threat T5, the cross-tenant leak). So how likely is a collision? We fix a tiny model — H hash buckets, n tenants dropped in by hash — derive the exact collision probability, derive the famous $1 - e^{-n^2/(2H)}$ approximation from it, and then show numerically why a *small* keyspace collides shockingly fast while the structured `(tenant, user)` tuple (effectively $H = 2^{64}$) makes collisions negligible. Every number is reproduced by the program in Appendix A.

Where this sits. This is **Topic 1 of Module 2.11** (Memory, Guardrails, Safety) — the math behind the memory-architecture rule and threat T5 “Cross-Tenant State Leak.” Its companions: Module 2.7 (tenant-namespaced thread IDs — the *working-memory* version of this same discipline) and Module 2.8 (subgraph-private short-term state).

Concepts covered, in order: the namespace as a hash bucket · the no-collision product $\prod (1 - i/H)$ · the approximation $\ln(1 - x) \approx -x$ · the exponential form $1 - e^{-n^2/(2H)}$ · the linear tail $n^2/(2H)$ · the classic $H=365, n=23$ result · small vs. astronomical keyspaces · isolation, not encryption.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: tenants dropped into buckets

Long-term memory in Module 2.11 lives in a *separate store*, keyed by a namespace (`"memories"`, `tenant_id`, `user_id`). Suppose — as a naive engineer might — you do not use the full tuple but instead hash each tenant down to one of H fixed buckets (a sharded table, a fixed-width key, a mod over an integer id). Two tenants that hash to the *same* bucket now share a namespace: a write by one can be read by the other. That is exactly threat T5. The question is purely combinatorial: drop n tenants into H buckets uniformly at random; what is the chance that *some* two land together?

Quantity	Symbol	Value here
Number of distinct namespace buckets	H	varies (365, 10^6 , 2^{64})
Number of tenants hashed into them	n	varies
Prob. of <i>no</i> collision among n	$P_0(n)$	$\prod_{i=0}^{n-1} (1 - \frac{i}{H})$
Prob. of <i>at least one</i> collision	$P_c(n)$	$1 - P_0(n)$

Notation. “Collision” means two distinct tenants share one bucket — not that all tenants collide. $P_c(n)$ is the probability of *at least one* such clash. We assume the hash spreads tenants uniformly and independently over the H buckets, which is the best case; a bad hash only makes collisions *more* likely.

Intuition

The surprise of the birthday problem is that collisions need pairs, and the number of pairs grows like n^2 , not n . With n tenants there are $\binom{n}{2} \approx n^2/2$ pairs, and each pair has a $1/H$ chance of clashing. So you expect trouble once $n^2/2 \approx H$, i.e. once $n \approx \sqrt{2H}$ — the *square root* of the keyspace, not the keyspace itself. A million-bucket table is not safe for a million tenants; it is shaky at barely a *thousand*.

2 Step 1 — the exact no-collision product

Add tenants one at a time and ask each to miss every bucket already taken. The first tenant lands anywhere: probability 1. The second must avoid 1 occupied bucket out of H : probability $1 - \frac{1}{H}$. The i -th tenant (with i counted from 0) must avoid the i already-filled buckets: probability $1 - \frac{i}{H}$. These are independent given the earlier placements, so they multiply:

$$P_0(n) = \prod_{i=0}^{n-1} \left(1 - \frac{i}{H}\right), \quad P_c(n) = 1 - P_0(n). \quad (1)$$

Mini-example — this operation in isolation

Take the classic calendar, $H = 365$ days, and $n = 3$ people. $P_0(3) = 1 \cdot (1 - \frac{1}{365}) \cdot (1 - \frac{2}{365}) = 1 \cdot 0.99726 \cdot 0.99452 = 0.99180$, so $P_c(3) = 0.0082$ — under one percent. The product starts near 1 and each new factor shaves a little more off. The factors get *bigger* bites as i climbs, which is why the curve is slow at first and then snaps upward.

3 Step 2 — from product to $e^{-n^2/(2H)}$

The product in Eq. (1) is exact but opaque. Take logs and use the first-order approximation $\ln(1 - x) \approx -x$, valid when $x = i/H$ is small (large keyspace):

$$\begin{aligned} \ln P_0(n) &= \sum_{i=0}^{n-1} \ln\left(1 - \frac{i}{H}\right) \approx \sum_{i=0}^{n-1} \left(-\frac{i}{H}\right) = -\frac{1}{H} \sum_{i=0}^{n-1} i \\ &= -\frac{1}{H} \cdot \frac{n(n-1)}{2} \approx -\frac{n^2}{2H}. \end{aligned} \quad (2)$$

The same triangular number $\frac{n(n-1)}{2}$ that drives ReAct cost reappears here. Exponentiate to undo the log, and subtract from 1:

$$\boxed{P_c(n) \approx 1 - e^{-n^2/(2H)}} \quad (3)$$

And when the exponent itself is tiny ($n^2 \ll 2H$), $e^{-x} \approx 1 - x$ gives the cleanest tail of all:

$$P_c(n) \approx \frac{n^2}{2H}. \quad (4)$$

Mini-example — this operation in isolation

Check the approximation at the famous point: $H = 365$, $n = 23$. Then $n^2/(2H) = 529/730 = 0.7247$, so $P_c \approx 1 - e^{-0.7247} = 1 - 0.4846 = 0.5155$. The *exact* product in Eq. (1) gives 0.5073. ✓ The approximation is within one point of the exact value — and both cross one-half right around $n = 23$, the result that surprises everyone who first meets it.

4 Step 3 — a small keyspace collides surprisingly fast

Now apply this to memory. Suppose a tenant id is hashed into $H = 10^6$ buckets (“a million slots, surely enough”). The rule-of-thumb crossover is $n \approx 1.1774\sqrt{H}$, the n at which $P_c = \frac{1}{2}$. For $H = 10^6$ that is $n \approx 1177$. We tabulate P_c exactly from Eq. (1) and against the approximation Eq. (3):

tenants	n	P_c exact	P_c approx	reading
	10	0.000045	0.000050	effectively safe
	100	0.004938	0.004988	~0.5% — still rare
	1,000	0.393267	0.393469	already a 40% chance
	1,178	0.500189	0.500347	the coin-flip point
	2,000	0.864710	0.864665	near-certain clash
	5,000	0.999996	0.999996	a collision is guaranteed

A million-bucket keyspace is at a **coin-flip** for cross-tenant leakage by barely 1,200 tenants, and is effectively guaranteed to leak by 5,000 — a tenant count any real SaaS passes in its first year. Sample check, $n = 1000$: $n^2/(2H) = 10^6/(2 \times 10^6) = 0.5$, so $P_c \approx 1 - e^{-0.5} = 1 - 0.6065 = 0.3935$. ✓ Matches the table to four places.

Watch out

The danger is the gap between intuition and the square root. “A million buckets for a thousand tenants” *sounds* like a 1000× safety margin. It is not: $\sqrt{10^6} = 1000$, so a thousand tenants is exactly where the margin runs out. Sizing a keyspace by n instead of n^2 is the single most common way namespaces silently start sharing.

5 Step 4 — the structured tuple makes H astronomical

The module’s fix is not a bigger hash table — it is to stop hashing into a small space at all. Namespacing by the full (**tenant_id**, **user_id**) *tuple* means the key is the identity itself: distinct tenants occupy distinct, *structurally separated* keys, with no modular folding to clash over. If you must reason about it as a hash (e.g. a 64-bit fingerprint), the keyspace is $H = 2^{64} \approx 1.8 \times 10^{19}$. Plug realistic n into the linear tail Eq. (4):

tenants	n	$P_c \approx n^2/(2H), \quad H = 2^{64}$	reading
	10^3	2.71×10^{-14}	1 in 37 trillion
	10^6	2.71×10^{-8}	1 in 37 million
	10^9	2.71×10^{-2}	only here (10^9 tenants!) does it stir

At a million tenants the chance of *any* pair sharing a 64-bit key is about 2.7×10^{-8} — a one-in-thirty-seven-million event, versus the *coin flip* a million-bucket table gave us. Sample check, $n = 10^6$: $n^2 = 10^{12}$, $2H = 3.689 \times 10^{19}$, ratio = 2.711×10^{-8} . ✓ Same number, derived two ways.

Intuition

Collisions are a **keyspace-sizing** problem, and the cure is a **large, structured key**. Each extra bit of key *doubles* H , which *quarters* the per-tenant collision probability (because $P_c \propto n^2/H$). Going from 20 bits ($H \approx 10^6$) to 64 bits buys $2^{44} \approx 1.8 \times 10^{13}$ times more headroom — the difference between “leaks this year” and “never.”

6 Step 5 — isolation, not encryption

It is worth stating plainly what this math does and does *not* say, because the two are easy to confuse:

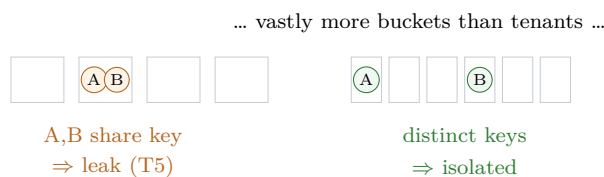
- Namespacing prevents leakage by **isolation** — tenant A and tenant B write to and read from *separate keys*, so A’s data never enters B’s retrieval scope. The separation is structural, enforced by the store’s key lookup.
- It is **not encryption**. There is no secret here. The (tenant, user) tuple is not hidden; it is simply *unique*. A large, structured key keeps tenants apart the way distinct street addresses keep mail apart — not the way a lock keeps a thief out.

So the engineering lever is keyspace *size and structure*, full stop. A collision is the failure of isolation, and the only knobs are: how many distinct keys exist (H), and how many identities you pour in (n). Make H astronomically large *by construction* — the full tuple, not a small hash — and $n^2/(2H)$ vanishes for any n you will ever see.

7 The mechanism, drawn

small H

tuple key $H = 2^{64}$



Left: a few buckets, so two tenants collide and share a namespace — the leak. Right: the structured tuple gives so many more keys than tenants that each sits alone. The fix is more buckets, not a secret.

8 Collision cheat-sheet

No collision among n (exact)	$P_0(n) = \prod_{i=0}^{n-1} \left(1 - \frac{i}{H}\right)$
At least one collision	$P_c(n) = 1 - P_0(n)$
Log approximation ($\ln(1-x) \approx -x$)	$\ln P_0 \approx -\frac{n(n-1)}{2H} \approx -\frac{n^2}{2H}$
Exponential form	$P_c(n) \approx 1 - e^{-n^2/(2H)}$
Tiny-exponent tail	$P_c(n) \approx \frac{n^2}{2H}$
50% crossover	$n_{1/2} \approx 1.1774 \sqrt{H}$
Each +1 key bit	doubles H , quarters P_c

9 An honest word about these numbers

The keyspaces here (365 , 10^6 , 2^{64}) are illustrative anchors: a calendar for intuition, a plausibly-naive million-slot table, and a 64-bit fingerprint for the structured tuple. Real systems rarely hash tenants into a fixed small space on purpose — but they do it *by accident*: a mod over an integer id, a fixed-width shard key, a truncated hash, a reused session-id field. What is *not* illustrative is the structure. The collision probability is governed by $n^2/(2H)$: it grows with the *square* of the population and falls with the keyspace, so safety is bought at the *square root* of H , never at H itself. Change the numbers and the table moves; the $\sqrt{H}$ law and the isolation-not-encryption framing do not. That is the thing to carry into every memory store you namespace.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the classic birthday check, the small-keyspace table, and the 2^{64} tuple table; change H to size your own namespace.

```

1  # namespace_collision.py -- reproduces every number in
2  # "Namespace Collision Probability, by Hand".
3  import math
4
5  def p_no_collision(n, H): # exact product, Eq. (1)
6      p = 1.0
7      for i in range(n):
8          p *= (1.0 - i / H)
9      return p
10
11 def p_collision_exact(n, H):
12     return 1.0 - p_no_collision(n, H)
13
14 def p_collision_approx(n, H): # 1 - exp(-n^2/(2H)), Eq. (4)
15     return 1.0 - math.exp(-(n * n) / (2.0 * H))
16
17 def p_collision_linear(n, H): # n^2/(2H), Eq. (5)
18     return (n * n) / (2.0 * H)
19
20 # --- classic birthday, H=365 ---
21 print(f"H=365, n=23: exact={p_collision_exact(23,365):.4f} "
22       f"approx={p_collision_approx(23,365):.4f}")
23

```

```

24 # --- small keyspace H=1e6 ---
25 H = 10**6
26 for n in (10, 100, 1000, 1178, 2000, 5000):
27     print(f"H=1e6 n={n:5d}: exact={p_collision_exact(n,H):.6f} "
28           f"approx={p_collision_approx(n,H):.6f}")
29 print(f"50% crossover n ~ 1.1774*sqrt(H) = {1.1774*math.sqrt(H):.1f}")
30
31 # --- structured tuple H=2^64 (linear tail; exact product is wasteful here) ---
32 H = 2**64
33 for n in (10**3, 10**6, 10**9):
34     print(f"H=2^64 n={n:>13,}: P_c ~ n^2/(2H) = {p_collision_linear(n,H):.3e}")
35
36 # Expected output:
37 # H=365, n=23: exact=0.5073 approx=0.5155
38 # H=1e6 n= 10: exact=0.000045 approx=0.000050
39 # H=1e6 n= 100: exact=0.004938 approx=0.004988
40 # H=1e6 n= 1000: exact=0.393267 approx=0.393469
41 # H=1e6 n= 1178: exact=0.500189 approx=0.500347
42 # H=1e6 n= 2000: exact=0.864710 approx=0.864665
43 # H=1e6 n= 5000: exact=0.999996 approx=0.999996
44 # 50% crossover n ~ 1.1774*sqrt(H) = 1177.4
45 # H=2^64 n= 1,000: P_c ~ n^2/(2H) = 2.711e-14
46 # H=2^64 n= 1,000,000: P_c ~ n^2/(2H) = 2.711e-08
47 # H=2^64 n=1,000,000,000: P_c ~ n^2/(2H) = 2.711e-02

```

— end of document —